

Securing Your Containers: Best Practices for Docker and Kubernetes

According to a recent survey, 94% of organisations adopting container technology are prioritising security to protect their applications and data. Here are important best practices for securing containers to help developers mitigate risks and protect their applications from potential threats.



Container security refers to the measures and practices aimed at securing containerised applications throughout their lifecycle. With the unique challenges that containers present—such as shared operating systems and dynamic environments—understanding these security concerns is vital. The shared responsibility model highlights that security is a joint effort between the host and the container. While the host environment must be secured, developers also need to ensure their containers are robust against vulnerabilities.

The unique challenges of container security

Container security presents unique challenges that differ significantly from traditional application security. Understanding these challenges is essential for developers aiming to protect their containerised applications effectively.

Shared operating system

vulnerabilities: One of the primary challenges of container security is that multiple containers run on the same underlying operating system kernel. This shared architecture can lead to vulnerabilities that, if exploited in one container, may affect others. For instance, if a malicious actor manages to gain access to a vulnerable container running as root, they can potentially exploit the kernel vulnerability to access the host system or other containers.

Real-life scenario: In 2019, a major security breach was reported where attackers exploited a vulnerability in the Docker daemon to gain elevated privileges on the host machine. The attackers were able to create a new container that had full access to the host, leading to a significant data breach in the affected organisation. This incident highlighted the risks associated with container isolation when the underlying host is compromised.

Dynamic environment risks:

Containers are designed to be ephemeral and can be created, destroyed, and modified on-the-fly. This dynamic nature poses security challenges, as it becomes difficult to maintain consistent security policies across environments. The rapid scaling of containers can lead to situations where outdated or unpatched images are deployed, introducing vulnerabilities into production environments.

Use case: Consider a development team that frequently deploys new versions of their application using continuous integration and continuous deployment (CI/CD) pipelines. If the CI/CD pipeline is not configured to scan images for vulnerabilities before deployment, a container running an outdated library could inadvertently be pushed to production, exposing the application to known exploits.

Lack of visibility and control: The distributed nature of containerised applications can lead to a lack of visibility into network traffic and application behaviour. Containers often communicate with each other over the network, and without proper monitoring and logging in place, it becomes challenging to detect and respond to suspicious activities. This lack of visibility can hinder incident response efforts and complicate compliance requirements.

Real-life scenario: In 2020, a financial institution faced significant